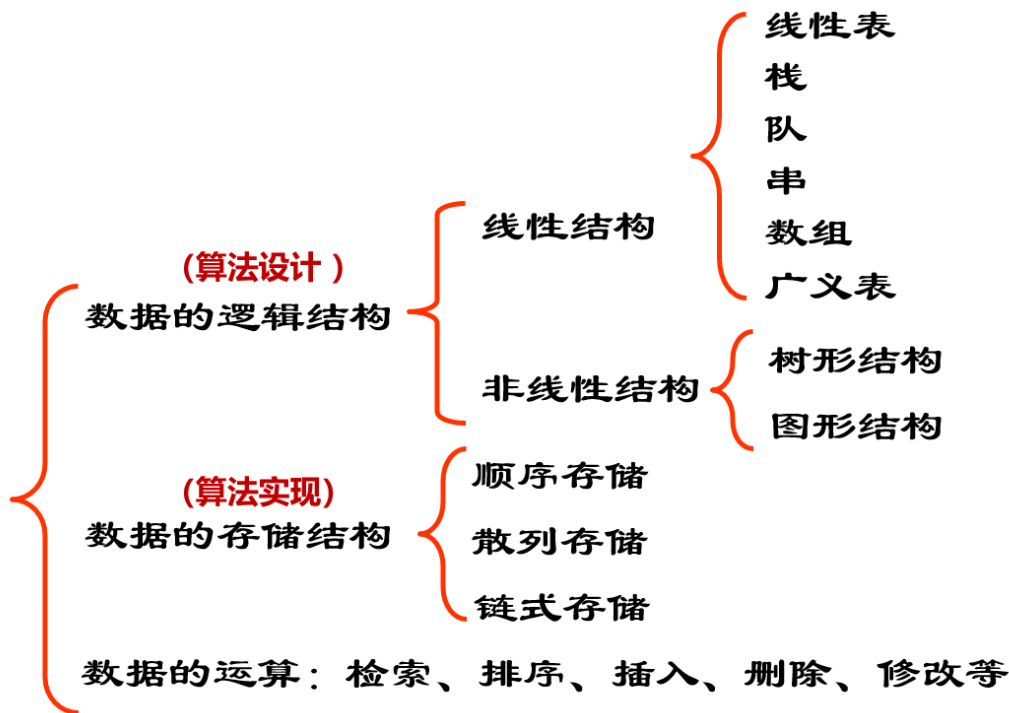


数据结构

1 绪论

1.1 数据结构总览



1.2 时间空间复杂度的计算

2 线性表

2.1 线性表基本结构及操作

```
ADT  LinearList{
    数据元素: D = { ai | ai ∈ D0, i=1,2,..., n    n ≥ 0 , D0为某一数据对象 }
    关系:      S = { <ai,ai+1> | ai, ai+1 ∈ D0, i=1,2, ...,n-1 }
    基本操作:
    1) InitList(L)      操作前提: L为未初始化线性表。操作结果: 将L初始化为空表。
    2) DestroyList(L)   操作前提: 线性表L已存在。 操作结果: 将L销毁。
    3) ClearList(L)     操作前提: 线性表L已存在 。操作结果: 将表L置为空表。
    .....
}ADT  LinearList
```

2.2 顺序表结构及操作

2.2.1 结构

```
#define MAXSIZE 100          /*线性表可能达到的最大长度*/
typedef struct
{
    ElemType elem[MAXSIZE];    /*线性表占用的数组空间*/
    int last;
    /*记录线性表中最后一个元素在数组elem[ ]中的位置(下标值),空表置为-1。注意,数组中实际存储
    的元素个数为 last+1 */
} SeqList;
/*注意区分元素的序号和数组的下标,如a1的序号为1,而其对应的数组下标为0。*/
```

2.2.2 查找

```
int Locate(SeqList L, ElemType e) {
    i=0 ;          /*i为扫描计数器,初值为0,即从第一个元素开始比较*/
    while ((i<=L.last)&&(L.elem[i]!=e) )
        i++;      /*顺序扫描表,直到找到值为e的元素,或扫描到表尾而没找到*/
    if (i<=L.last)
        return(i);    /*若找到值为e的元素,则返回其序号*/
    else
        return(-1);   /*若没找到,则返回空序号*/
}
/*算法的时间复杂度为O(n)*/
```

2.2.3 插入

```
#define OK 1
#define ERROR 0
int InsList(SeqList *L,int i,ElemType e) {
    int k;
    if( (i<1) || (i>L->last+2) ) {    /*首先判断插入位置是否合法*/
        printf("插入位置i值不合法");
        return(ERROR);    }
    if( L->last>=MAXSIZE-1) {
        printf("表已满无法插入");
        return(ERROR);    }
    for( k=L->last;k>=i-1;k--)    /*为插入元素而移动位置*/
        L->elem[k+1]=L->elem[k];
    L->elem[i-1]=e;    /*在C语言中数组第i个元素的下标为i-1*/
    L->last++;
    return(OK);
}
```

插入的本质就是将该位置往后的所有数据向后平移一格。

数据结构的题目中很重要的一点就是**边界**判断,包括**数据**是否合法,**位置**是否合法等。

当对结构进行更改时还要注意对**参数**更改,例如**线性表的长度参数**。

2.2.4 删除

```
/*在顺序表L中删除第i个数据元素，并用指针参数e返回其值*/
int DelList(SeqList *L,int i, ElemType *e) {
    int k;
    if((i<1)|| (i>L->last+1)) {
        printf("删除位置不合法！");
        return(ERROR);
    }
    *e = L->elem[i-1];                /* 将删除的元素存放到e所指向的变量中*/
    for(k=i;k<=L->last;k++)
        L->elem[k-1]= L->elem[k];    /*将后面的元素依次前移*/
    L->last--;
    return(OK);
}
```

同样的,在删除时,本质上就是将后面的数据向前平移一格。

这里也不能忘记删除边界判断和更改**长度参数**!

2.2.5 合并 *

```
void merge(SeqList *LA, SeqList *LB, SeqList *LC){
    int i,j,k,l;
    i=0;j=0;k=0;
    while(i<=LA->last&& j<=LB->last)    /* 只要有一个表被遍历了就结束循环 */
        if(LA->elem[i]<=LB->elem[j]) {
            LC->elem[k]= LA->elem[i];
            i++; k++; }
        else {
            LC->elem[k]=LB->elem[j];
            j++; k++; }
    while(i<=LA->last) {                /* 当表LA有剩余元素，则将余下的元素赋给表LC */
        LC->elem[k]= LA->elem[i];
        i++; k++; }
    while(j<=LB->last) {                /* 当表LB有剩余元素，则将余下的元素赋给表LC */
        LC->elem[k]= LB->elem[j];
        j++; k++; }
    LC->last=LA->last+LB->last+1;
}
```

这里有几个注意点:

1. 只用遍历完一个表,把另一个表剩下的部分直接接在后面就OK.
2. i,j,k三个参数分别表示在三个表中的位置,结束时要记得更新LC长度参数为k.

2.3 线性表的链式储存

2.3.1 链表的结构

```
typedef struct Node {           /* 结点类型定义 */
    ElemType data;
    struct Node *next;
}Node, *LinkedList;           /* LinkedList为结构指针类型*/
```

LinkedList 和 **Node** 同为结构指针类型

LinkedList 一般用作单链表的头指针变量

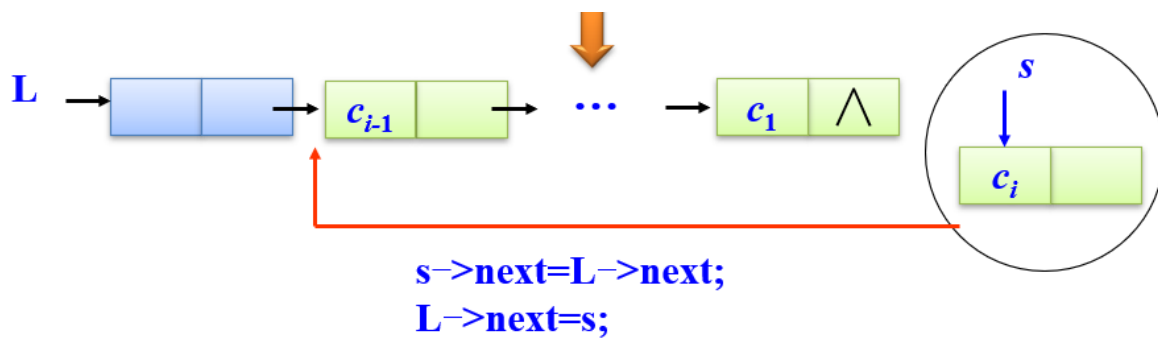
Node 一般用作指向单链表中结点的指针

```
*L = (LinkedList)malloc(sizeof(Node));
```

上方代码是一个申请空节点的操作,注意**malloc**函数的运用,L是**LinkedList**类型的指针,大小为**sizeof(node)**.
一般使用的时候指针变量类型都是**LinkedList**.

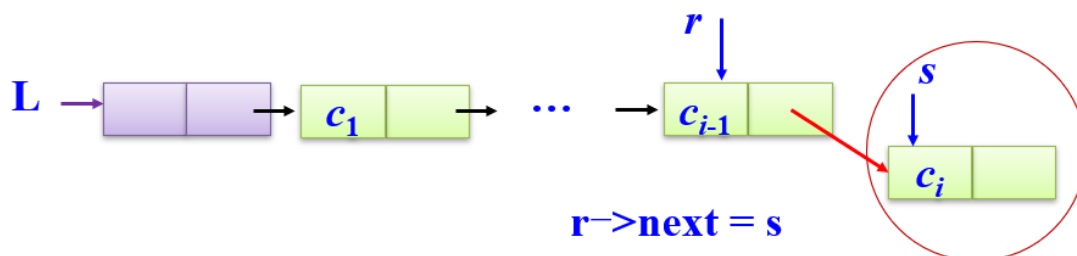
2.3.2 头插法创建链表

```
void CreateFromHead(LinkedList L) {
    Node *s;
    char c;
    int flag=1;
    while(flag) {                /* flag初值为1, 当输入"$"时, 置flag为0, 建表结束*/
        c=getchar();
        if(c!='$') {
            s=(Node*)malloc(sizeof(Node)); /*建立新结点s*/
            s->data=c;
            s->next=L->next;              /*将s结点插入表头*/
            L->next=s;
        }
        else
            flag=0;
    }
}
```



2.3.3 尾插法创建链表

```
void CreateFromTail(LinkList L){ /* flag初值为1, 当输入"$"时, flag为0, 建表结束 */
    Node *r, *s;
    char c;
    int flag = 1;
    r = L; /* r 指针动态指向链表的当前表尾 */
    while(flag)
        c = getchar();
        if(c != '$') {
            s = (Node*)malloc(sizeof(Node));
            s->data = c;
            r->next = s;
            r = s;
        }
        else {
            flag = 0;
            r->next = NULL; /* 将最后一个结点的next链域置为空, 表示链表的结束 */
        }
    }
}
```



2.3.4 头插法与尾插法的比较

头插法是锁定一个**尾节点**,在**尾节点的前面**挨个插入,因此生成的是**倒序**.

尾插法是锁定一个**头节点**,在**头节点的后面**挨个插入,因此生成的是**顺序**.

2.4 循环链表

```
void initClinklist(LinkList *CL) { /* CL是循环单链表的头指针变量 */
    *CL = (LinkList)malloc(sizeof(Node)); /* 建立头结点 */
    (*CL) -> next = *CL ; /*置为空表*/
}
void createClinklist (LinkList CL){/* CL已经初始化，当输入$时，建表结束 */
    Node *rear, *s;
    char c;
    rear = CL; /* rear 指针动态指向链表的当前表尾 */
    c=getchar(); /* 读入第一个元素 */
    while(c!='$') {
        s = (Node*)malloc(sizeof(Node));
        s->data = c;
        rear->next = s;
        rear = s;
        c=getchar();
    }
    rear->next = CL; /*将最后一个结点的next链域指向头结点*/
}
```

3 栈和队列

3.1 栈

3.1.1 栈的基本结构

```
#define Stack_Size 50
typedef struct{
    StackElementType elem[Stack_Size]; /*用来存放栈中元素的一维数组*/
    int top; /*用来存放栈顶元素的下标，top为-1表示空栈*/
}SeqStack;
```

栈是一种只能在一端进行插入或删除操作的线性表。

栈只能选取同一个端点进行插入和删除操作

3.1.2 栈的基本操作

```
int IsEmpty(SeqStack *S) { /*判栈S为空栈时返回值为真，反之为假*/
    return(S->top==-1?TRUE:FALSE);
}

int IsFull(SeqStack *S) { /*判栈S为满时返回真，否则返回假*/
    return(S->top== Stack_Size-1? TRUE: FALSE);
}

int Push(SeqStack * S, StackElementType x) {
    if(S->top== Stack_Size-1)
        return(FALSE); /*栈已满*/
    S->top++;
```

```

    S->elem[S->top]=x;
    return(TRUE);
}
int Pop(SeqStack *S, StackElementType *x){
    if(S->top== -1)    /*栈为空*/
        return(FALSE);
    else {
        *x= S->elem[S->top];
        S->top--;    /* 修改栈顶指针 */
        return(TRUE);
    }
}
int GetTop (SeqStack *S, StackElementType *x) {
    /* 将栈S的栈顶元素弹出，放到x所指的存储空间中，但栈顶指针保持不变 */
    if(S->top== -1)    /*栈为空*/
        return(FALSE);
    else {
        *x = S->elem[S->top];
        return(TRUE);
    }
}
}

```

3.2 队列

3.2.1 队列的基本定义

队列只能选取一个端点进行插入操作，另一个端点进行删除操作

把进行插入的一端称做**队尾**（rear）。

进行删除的一端称做**队首**（front）。

向队列中插入新元素称为**入队**，新元素入队后就成为**新的队尾元素**。

从队列中删除元素称为**出队**，元素出队后，其**后继**元素就成为**队首元素**。

队列具有**先进先出** (Fist In Fist Out，缩写为**FIFO**)的特性

对比:栈具有**先进后出**的特点(**FILO**)

```

typedef struct Node{
    QueueElementType data;    /*数据域*/
    struct Node *next;    /*指针域*/
}LinkQueueNode;

typedef struct {
    LinkQueueNode *front;
    LinkQueueNode *rear;
}LinkQueue;

```

3.2.2 队列的基本操作

```
int InitQueue(LinkQueue * Q) { /* 将Q初始化为一个空的链队列 */
    Q->front = (LinkQueueNode *)malloc(sizeof(LinkQueueNode));
    if(Q->front!=NULL){
        Q->rear = Q->front;
        Q->front->next = NULL;
        return(TRUE);
    }
    else
        return(FALSE);          /* 溢出! */
}

int EnterQueue(LinkQueue *Q, QueueElementType x){ /* 将数据元素x插入到队列Q中 */
    LinkQueueNode * NewNode;
    NewNode=(LinkQueueNode * )malloc(sizeof(LinkQueueNode));
    if(NewNode!=NULL) {
        NewNode->data=x;
        NewNode->next=NULL;
        Q->rear->next=NewNode;
        Q->rear=NewNode;
        return(TRUE);
    }
    else return(FALSE);          /* 溢出! */
}

int DeleteQueue(LinkQueue * Q, QueueElementType *x) {
    LinkQueueNode * p;
    if(Q->front==Q->rear)
        return(FALSE);
    p=Q->front->next;
    Q->front->next=p->next; /* 队头元素p出队 */
    if(Q->rear==p)          /* 如果队中只有一个元素p, 则p出队后成为空队 */
        Q->rear=Q->front;
    *x=p->data;
    free(p);                /* 释放存储空间 */
    return(TRUE);
}
```

在数据结构代码的书写过程中,要注意**释放存储空间**!

3.2.3 特殊队列--循环队列

```
#define MAXSIZE 50 /*队列的最大长度*/
typedef struct{
    QueueElementType element[MAXSIZE];
                                /* 队列的元素空间*/
    int front; /*头指针指示器*/
    int rear ; /*尾指针指示器*/
}SeqQueue;
```

把数组的**前端**和**后端**连接起来, 形成一个**环形的顺序表**, 即把存储队列元素的表从逻辑上看成一个环, 称为**循环队列**或**环形队列**。

队空和队满的情况: rear == front

4 串

串的逻辑结构和线性表极为相似，区别仅在于串的数据对象约束为字符集。

串的基本操作和线性表有很大差别：

在线性表的基本操作中，大多以“单个元素”作为操作对象；

在串的基本操作中，通常以“串的整体”作为操作对象。

4.1 串的模式匹配

Brute Force算法

```
/*求从主串s的下标pos起，串t第一次出现的位置，成功返回位置序号，不成功返回-1*/
int StrIndex(SString s, int pos, SString t) {
    int i, j, start;
    if (t.len == 0)
        return(0);          /* 模式串为空串时，是任意串的匹配串 */
    start = pos;
    i = start;
    j = 0;                   /* 主串从pos开始，模式串从头（0）开始 */
    while (i < s.len && j < t.len)
        if (s.ch[i] == t.ch[j]) {
            i++;
            j++;
        }                   /* 当前对应字符相等时推进 */
        else {
            start++;         /* 当前对应字符不等时回溯 */
            i = start;
            j = 0;           /* 主串从start+1开始，模式串从头（0）开始 */
        }
    if (j >= t.len)
        return(start);      /* 匹配成功时，返回匹配起始位置 */
    else
        return(-1);         /* 匹配不成功时，返回-1 */
}
```

4.2 KMP算法

[B站的KMP算法讲解\(甚至是学姐讲课\)](#)

5 数组与广义表

5.1 数组

5.1.1 计算数组元素的地址

例题:对一个二维数组 A'[6][8]' 按行优先存储, 若 A'[0][0]' 的地址为1000, 每个元素占4字节, 则 A'[3][5]' 的地址是1124.

5.1.2 稀疏矩阵三元组表表示

```
#define MAXSIZE 1000          /*非零元素的个数最多为1000*/
typedef struct{
    int row, col;              /*该非零元素的行下标和列下标*/
    ElementType e;             /*该非零元素的值*/
}Triple;
typedef struct{
    Triple data[MAXSIZE];      /* 非零元素的三元组表 */
    int m, n, len;             /*矩阵的行数、列数和非零元素的个数*/
}TSMatrix;
```

5.1.3 稀疏矩阵转置

```
/*把矩阵A转置到B所指向的矩阵中去。矩阵用三元组表表示*/
void TransposeTSMatrix(TSMatrix A, TSMatrix * B) {
    int i, j, k;
    B->m= A.n ; B->n= A.m ; B->len= A.len ;
    if(B->len>0) {
        j=0;                      /* j为三元组表B的下标 */
        for(k=0; k<A.n; k++)      /* 扫描三元组表A共n次 */
            for(i=0; i<A.len; i++) /* i为三元组表A的下标 */
                if(A.data[i].col==k){/* 寻找三元组表A的列值为k的进行转置 */
                    B->data[j].row=A.data[i].col;
                    B->data[j].col=A.data[i].row;
                    B->data[j].e=A.data[i].e;
                    j++;
                }                  /* 内循环if结束*/
    }                             /* if(B->len>0)结束*/
}
```

上述方法需要两次遍历,时间复杂度较高.

以下是一个快速的转置方法:

```
FastTransposeTSMatrix (TSMatrix A, TSMatrix * B) {
    int col, t, p, q;
    int num[MAXSIZE], position[MAXSIZE] ;
```

```

B->len= A.len ; B->n= A.m ; B->m= A.n ;
if(B->len) {
    for(col=0;col<A.n;col++)
        num[col]=0;          /*清零num数组*/
    for(t=0;t<A.len;t++)
        num[A.data[t].col]++; /*计算三元组表A每一列的非零元素的个数*/
    position[0]=0;
    for(col=1;col<A.n;col++) /*求col列中第一个非零元素在B.data[ ]中的正确位置*/
        position[col]=position[col-1]+num[col-1];
    for(p=0;p<A.len;p++) { /* 从头扫描三元组表A一次 */
        col=A.data[p].col;
        q=position[col]; /*col列中第一个非零元素在B.data[ ]中的正确位置*/
        B->data[q].row=A.data[p].col;
        B->data[q].col=A.data[p].row;
        B->data[q].e=A.data[p].e
        position[col]++; /*col列中下一个非零元素在B.data[ ]中的正确位置，修改了position数组*/
    }
}
}

```

这个转置方法原理如下:

$$\mathbf{A} = \begin{bmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

上图中的矩阵由A转置为B,我们需要建立两个数组num和position.

num数组存储每一列元素的个数,position数组根据num数组计算每一列第一个元素转置后在data数组的位置

每一列的元素进入data数组后,position数组中的元素自增.

col	0	1	2	3	4	5	6
num[col]	2	2	2	1	0	1	0
position[col]	0	2	4	6	7	7	8

5.2 十字链表

5.2.1 结构定义

```
typedef struct OLNode{
    int row, col;          /* 非零元素的行和列下标 */
    ElementType value;
    struct OLNode *down, *right; /* 非零元素所在行表列表的后继链域 */
}OLNode; *OLink;

typedef struct {
    OLink * row_head, *col_head; /* 行、列链表的头指针向量 */
    int m, n, len; /* 稀疏矩阵的行数、列数、非零元素的个数 */
}CrossList;
```

5.2.2 创建十字链表

```
/*采用十字链表存储结构，创建稀疏矩阵M*/
CreateCrossList (CrossList * M) {
    int m,n,t,i,j;
    OLink p,q;
    printf("输入M的行数、列数和非零元素的个数\n");
    scanf("%d,%d,%d",&m,&n,&t); /*输入M的行数、列数和非零元素的个数*/
    M->m=m; M->n=n; M->len=t;
    if(!(M->row_head=(OLink *)malloc(m*sizeof(OLink))))
        exit(OVERFLOW);
    if(!(M->col_head=(OLink *)malloc(n*sizeof(OLink))))
        exit(OVERFLOW);
    for(i=0,i<m,i++)
        M->row_head[i]=NULL; /*初始化行头指针向量*/
    for(j=0,j<n,j++)
        M->col_head[j]=NULL; /*初始化列头指针向量*/
    for(scanf(&i,&j,&e);i!=-1;scanf(&i,&j,&e)) {
        if(!(p=(OLNode *)malloc(sizeof(OLNode))))
            exit(OVERFLOW);
        p->row=i; p->col=j; p->value=e; /*生成结点*/
        if(M->row_head[i]==NULL)
            M->row_head[i]=p;
        else{ /*寻找行表中的插入位置*/
            q=M->row_head[i];
            while(q->right&&q->right->col<j)
                q=q->right; /*找到链表末尾插入*/
            p->right=q->right;
            q->right=p; /*完成插入*/
        }
        if(M->col_head[j]==NULL)
            M->col_head[j]=p;
        else{ /*寻找列表中的插入位置*/
            q=M->col_head[j];
            while(q->down&&q->down->row<i);
            q->down=p; /*找到链表末尾插入*/
        }
    }
}
```

```

        p->down=q->down;
        q->down=p;      /*完成插入*/
    }
} /*for结束*/
}

```

5.3 广义表

```

typedef enum {ATOM, LIST} ElemTag;    /*ATOM=0, 表示原子; LIST=1, 表示子表*/
typedef struct GLNode {
    ElemTag tag;      /*标志位tag用来区别原子结点和表结点*/
    union {
        AtomType atom; /*原子结点的值域atom*/
        struct {
            struct GLNode * hp, *tp;
        } htp;        /*表结点的指针域htp, 包括表头指针域hp和表尾指针域tp*/
    } atom_htp;      /* atom_htp 是原子结点的值域atom和表结点的指针域htp的联合体域*/
} GLNode, *GLList;

```